

AskSinAnalyzer auf Raspberry Pi mit Homematic-Funkmodul

Nachbau des HomeMatic-Funktelegramm-Analyzers auf Basis von Raspberry Pi + CC1101, inklusive Erweiterungen für Alarmer, Langzeit-Logging und ioBroker-Integration

Raspberry Pi

CC1101 868 MHz

ATmega328P Sniffer

AskSinAnalyzerXS (Node.js)

ioBroker

Homematic / RaspberryMatic

Grafana / InfluxDB

Erstellt für: Silvio
Datum: 26. Juli 2026
Basis-Projekt: jp112sdl/AskSinAnalyzer & psi-4ward/AskSinAnalyzerXS
Ziel-Zentrale: Homematic CCU / RaspberryMatic + ioBroker
Status: Konzept / Planungsdokument

Inhaltsverzeichnis

1	Management-Zusammenfassung (Kurzfaszit)
2	Was der AskSinAnalyzer macht
3	Systemarchitektur & Datenfluss
4	Machbarkeitsanalyse
5	Umsetzungsvarianten für den Raspberry Pi
6	Stückliste (BOM) & Kostenschätzung
7	Software-Stack
8	Projektplan in Phasen
9	Erweiterungskonzepte im Detail
10	Zeit- & Meilensteinplan
11	Risiken & Gegenmaßnahmen
12	Quellen & Referenzen

1 Management-Zusammenfassung (Kurzfaszit)

Das Vorhaben ist **gut und mit überschaubarem Aufwand machbar**. Der AskSinAnalyzer ist von Haus aus zweigeteilt: ein kleiner **Funk-Sniffer** (ATmega328P + CC1101-Modul) empfängt die Homematic-Funktelegramme und schickt sie über eine serielle Schnittstelle an eine **Analyzer-Software**. Für den Raspberry Pi existiert mit **AskSinAnalyzerXS** (Node.js/Docker) bereits eine ausdrücklich Linux- und Pi-taugliche Analyzer-Variante – der Betrieb auf dem Pi ist also kein „Hack“, sondern ein **dokumentierter, unterstützter Pfad**. Zusätzlich gibt es eine fertige **Aufsteckplatine für den Pi-GPIO-Header** (der-pw/AskSinAnalyzerXS-RPi), die den Sniffer ohne USB direkt auf den Pi setzt.

Fazit

Umsetzbar in geschätzt **2-4 Wochenend-Sessions** für den Grundaufbau, Materialkosten **ca. 50-120 €**. Alle vier gewünschten Erweiterungsrichtungen (zusätzliche Metriken/Alarmer, Langzeit-Logging & Dashboard, ioBroker-Integration, Hardware-Erweiterung) sind realistisch – besonders die ioBroker-Integration passt gut zu deinem Adapter-/Node-RED-Hintergrund.

Wichtigste Einschränkung

Der Analyzer dekodiert ausschließlich **klassische Homematic-Funktelegramme (BidCos/AskSin, 868 MHz)**. **Homematic IP (HmIP)** nutzt ein anderes Protokoll und wird **nicht** ausgewertet. Für ein reines HmIP-Setup ist das Werkzeug nur eingeschränkt sinnvoll (siehe Abschnitt 4).

2 Was der AskSinAnalyzer macht

Der AskSinAnalyzer ist ein „Funktelegramm-Dekodierer für den Einsatz in HomeMatic-Umgebungen“. Er hört den Funkverkehr passiv mit und wertet ihn aus, um Funk- und Duty-Cycle-Probleme sichtbar zu machen. Typische Anwendungsfälle:

- Duty-Cycle-Analyse:** Erkennen, welches Gerät wie viel Sendezeit verbraucht (der 1-%-Duty-Cycle ist die häufigste Ursache für „tote“ Aktoren).
- Störungssuche:** Aufspüren von Geräten in Reboot-/Sende-Schleifen oder von „Dauersendern“, die den Kanal zustellen.
- Funkqualität (RSSI/Noise):** Bewertung von Signalstärke und Grundrauschen, um Reichweiten- und Störquellenprobleme zu diagnostizieren.
- Telegramm-Mitschnitt:** Live-Liste aller empfangenen Telegramme mit Absender/Empfänger, Nachrichtentyp und Zeitstempel.

3 Systemarchitektur & Datenfluss

Die Lösung besteht aus zwei logischen Bausteinen, die über eine serielle Verbindung gekoppelt sind:

Baustein	Aufgabe	Umsetzung
1) Funk-Sniffer (AskSinSniffer328P)	Empfängt die 868-MHz-Homematic-Telegramme über das CC1101-Modul und gibt sie als Datenstrom über UART/Seriell aus.	ATmega328P (Arduino Pro Mini 3,3 V/8 MHz oder Nano) + CC1101-Funkmodul. Firmware = Sketch <code>AskSinSniffer328P</code> aus dem Repo.
2) Analyzer (Auswertung + Web-UI)	Liest den seriellen Strom, dekodiert die Telegramme, berechnet Duty-Cycle/RSSI, speichert Langzeitdaten und stellt die Web-Oberfläche bereit.	Für den Pi: AskSinAnalyzerXS (Node.js / Electron / Docker). Web-UI unter <code>http://<pi>:8081</code> .

Datenfluss: Homematic-Geräte → 868 MHz Funk → CC1101 → ATmega328P (Sniffer) → UART/Seriell → Raspberry Pi (AskSinAnalyzerXS) → Web-UI / Datenbank / Integrationen

Namensauflösung

Damit in der Auswertung statt kryptischer Seriennummern die echten Gerätenamen erscheinen, kann AskSinAnalyzerXS die Geräteliste von einer **CCU** / **RaspberryMatic** oder von **FHEM** abrufen. Das passt direkt zu deinem Setup (CCU/RaspberryMatic vorhanden).

4 Machbarkeitsanalyse

4.1 Warum der Raspberry Pi hier besonders gut passt

- **Offizieller Pi-Pfad vorhanden:** AskSinAnalyzerXS unterstützt „Linux und Raspberry Pi“ ausdrücklich und lässt sich als Debian-/systemd-Dienst dauerhaft betreiben.
- **Dauerbetrieb statt Bastel-Gerät:** Anders als die ESP32-Variante mit TFT eignet sich der Pi ideal als 24/7-Monitoring-Station mit Langzeitspeicher.
- **Fertige Aufsteckplatine:** `der-pw/AskSinAnalyzerXS-RPi` setzt den 328P-Sniffer direkt auf den GPIO-Header (kein USB-Adapter nötig; Gerber-/KiCad-Dateien verfügbar).
- **Ökosystem:** Auf dem Pi laufen ioBroker, Node-RED, InfluxDB und Grafana problemlos parallel – ideale Basis für die geplanten Erweiterungen.

4.2 Technische Randbedingungen

Thema	Bewertung
Frequenz / Protokoll	868 MHz, klassisches Homematic BidCos/AskSin . HmIP wird nicht dekodiert.
Empfänger	CC1101 (868-MHz-Variante). Die Antenne muss zur Frequenz passen (868-MHz-Wendel-/SMA-Antenne), nicht die oft beiliegende 433-MHz-Antenne verwenden.
Serielle Kopplung	Über USB-UART-Adapter (<code>/dev/ttyUSB0</code>) <i>oder</i> direkt über die Pi-Hardware-UART (<code>/dev/serial0</code> , GPIO14/15). Baudrate gemäß Sniffer-Sketch (typisch 57600 Baud) in der XS-Konfiguration einstellen.
Rechenlast	Sehr gering. Ein Pi Zero 2 W oder Pi 3/4 reicht locker; bei zusätzlichem InfluxDB+Grafana+ioBroker auf demselben Gerät besser Pi 4 (2 GB+).
Löt-/Elektronikaufwand	Moderat: CC1101 an den 328P per SPI verdrahten und den Sketch flashen. Mit fertiger PCB (stan23 / der-pw) minimal.

4.3 Grenzen & offene Punkte

Nur ein Kanal / eine Frequenz gleichzeitig

Ein einzelner CC1101 empfängt auf einer Frequenz. Homematic-Klassik sendet primär auf 868,3 MHz – das ist abgedeckt. Vollständige „lückenlose“ Erfassung bei sehr hohem Funkaufkommen kann jedoch nie zu 100 % garantiert werden (ein passiver Sniffer kann kollidierende Telegramme verpassen).

- **HmIP:** Wird nicht ausgewertet – falls dein Netz überwiegend HmIP ist, sinkt der Nutzen.
- **Keine offizielle REST-/MQTT-Schnittstelle** in XS dokumentiert – für die ioBroker-Anbindung werden daher WebSocket/Node-RED bzw. die Datenbank oder ein eigener Adapter genutzt (siehe 9.3).
- **Pi-UART-Konfiguration** erfordert etwas Sorgfalt (serielle Konsole deaktivieren, ggf. Bluetooth-UART umlegen) – in Phase 2 abgedeckt.

5 Umsetzungsvarianten für den Raspberry Pi

Variante	Beschreibung	Vor-/Nachteile
A - USB-Sniffer (empfohlen für den Start)	328P-Sniffer + CC1101 auf Breadboard/Lochraster, per USB-UART-Adapter an den Pi (<code>/dev/ttyUSB0</code>).	+ Einfachste Verkabelung, Sniffer separat platzierbar (bessere Antennenlage). – Zusätzlicher USB-Adapter, „fliegender“ Aufbau.
B - GPIO-Aufsteckplatine	Fertige PCB (der-pw) steckt auf den GPIO-Header, Kopplung über die Pi-Hardware-UART.	+ Kompakt, sauber, kein USB-Adapter. – PCB fertigen/bestücken, Pi-UART muss konfiguriert werden, Antenne nah am Pi (Störungen).
C - Fertige/vorbestückte Module	Nano + CC1101-Adapterplatinen aus der Community (z. B. stan23-PCB) nutzen.	+ Weniger Lötarbeit. – Beschaffung/Verfügbarkeit variabel.

Empfehlung: **Mit Variante A starten** (schnell lauffähig, leicht zu debuggen), später optional auf Variante B für den sauberen Dauereinbau wechseln.

6 Stückliste (BOM) & Kostenschätzung

Komponente	Menge	Richtpreis	Hinweis
Raspberry Pi (Zero 2 W / 3B+ / 4 2GB)	1	15-60 €	Pi 4, falls ioBroker+Grafana mit drauf laufen sollen
microSD-Karte (32 GB, A1/A2)	1	7-12 €	Für Langzeit-DB lieber 32 GB+
Netzteil (offiziell)	1	8-12 €	Stabile Versorgung wichtig
CC1101-Funkmodul 868 MHz	1	3-8 €	Unbedingt 868-MHz-Variante
ATmega328P-Board (Pro Mini 3,3 V/8 MHz oder Nano)	1	3-6 €	3,3 V wegen CC1101-Pegel
USB-UART-Adapter (CP2102/FTDI)	1	3-5 €	Für Variante A + zum Flashen
868-MHz-Antenne (Wendel/SMA)	1	2-6 €	Reichweite steht/fällt hiermit
Breadboard/Lochraster, Jumper, Widerstand 330 Ω, LED	Set	5-8 €	Status-LED optional
Optional: PCB (stan23 / der-pw) fertigen lassen	1 Set	10-20 €	Meist 5er-Set beim Fertiger
Optional: Gehäuse (3D-Druck/Kauf)	1	5-15 €	Für Dauereinbau
Optional: kleines OLED-Display	1	4-7 €	RSSI/Noise-Anzeige am Sniffer
Summe (Grundaufbau, Variante A)		ca. 50-120 €	

Preise sind grobe Richtwerte (Endkundenhandel/Fernost-Versand) und können schwanken. Wenn Pi und microSD schon vorhanden sind, liegt der reine Zusatz-Materialaufwand bei ca. 20-45 €.

7 Software-Stack

Ebene	Komponente
Sniffer-Firmware	Arduino-Sketch <code>AskSinSniffer328P</code> (aus <code>jp112sdl/AskSinAnalyzer</code>), geflasht via Arduino IDE
Betriebssystem Pi	Raspberry Pi OS Lite (64-bit) – headless, als Dienst
Analyzer	AskSinAnalyzerXS (Node.js) – Installation via <code>npm i -g asksin-analyzer-xs</code> oder Docker
Namensauflösung	Anbindung an CCU/RaspberryMatic (oder FHEM) für Geräteliste
Integration (Erweiterung)	Node-RED / ioBroker (node-red- oder javascript-Adapter), optional eigener ioBroker-Adapter
Langzeit-Logging (Erweiterung)	InfluxDB + Grafana (nativ oder über ioBroker-Adapter)

```
# AskSinAnalyzerXS - zwei Installationswege

# a) Als globales npm-Paket (systemd-Dienst)
sudo npm i -g asksin-analyzer-xs
# danach seriellen Port + Baudrate in der Konfiguration eintragen

# b) Als Docker-Container
docker run -d --name analyzer --restart unless-stopped \
  -p 8081:8081 \
  -v $PWD/data:/data \
  --device=/dev/ttyUSB0 \
  psitrax/asksinanalyzer
# Web-UI: http://<raspberry-pi>:8081
```

8 Projektplan in Phasen

Phase 0 - Vorbereitung & Entscheidung · ~0,5 Tag

- Bestandsaufnahme: Anteil Homematic-Klassik vs. HmIP im eigenen Netz prüfen (Nutzen bestätigen).
- Variante wählen (A USB-Sniffer empfohlen für Start).
- Standort für den Sniffer festlegen (zentral, störungsarm, gute Antennenlage).
- Stückliste bestellen (Abschnitt 6).

Phase 1 - Sniffer bauen & flashen · ~1 Tag

- CC1101 per SPI an den ATmega328P verdrahten (autoritative Pinbelegung: Wiki-Seite *Elektronik_Verdrahtung*). Standard-SPI-Belegung siehe Tabelle unten.
- Arduino IDE einrichten, benötigte Bibliotheken laden, Sketch `AskSinSniffer328P` kompilieren.
- Sniffer flashen (bei Pro Mini 3,3 V über USB-UART-Adapter, DTR für Auto-Reset).

- Funktionstest: serielle Ausgabe am PC prüfen – kommen Telegramme an?

CC1101-Pin	Funktion	ATmega328P (Pro Mini/Nano)
SCK	SPI Clock	D13
MISO / SO	SPI Data out	D12
MOSI / SI	SPI Data in	D11
CSN / CS	Chip Select	D10
GDO0	Interrupt/Empfang	D2
VCC / GND	Versorgung 3,3 V	VCC (3,3 V) / GND

⚠ Verbindliche Pinbelegung immer der Repo-/Wiki-Doku entnehmen – je nach Board/Sketch-Version kann GDO0 abweichen. CC1101 ist ein 3,3-V-Baustein: keine 5-V-Pegel anlegen.

Phase 2 - Raspberry Pi Basis-Setup · ~0,5 Tag

- Raspberry Pi OS Lite (64-bit) headless flashen (SSH + WLAN vorkonfigurieren).
- System aktualisieren, feste IP/Hostname vergeben.
- **Variante A:** USB-UART-Adapter anstecken, Port ermitteln (`/dev/ttyUSB0`).
- **Variante B (GPIO):** serielle Konsole deaktivieren, UART aktivieren (`enable_uart=1`), auf Pi 3/4 Bluetooth-UART umlegen, damit `/dev/serial0` die vollwertige PL011-UART nutzt.

```
# GPIO-UART auf dem Pi vorbereiten (nur Variante B)
sudo raspi-config      # Interface Options -> Serial:
                        #   Login shell over serial = NO
                        #   Serial hardware enabled  = YES

# /boot/firmware/config.txt ergänzen:
#   enable_uart=1
#   dtoverlay=disable-bt      # Pi 3/4: volle UART auf GPIO14/15
sudo systemctl disable hciuart
sudo reboot
```

Phase 3 - AskSinAnalyzerXS installieren & Grundbetrieb · ~0,5 Tag

- Node.js installieren bzw. Docker einrichten; XS per npm oder Container starten (Abschnitt 7).
- Seriellen Port + Baudrate konfigurieren; als systemd-Dienst für Autostart einrichten.
- Web-UI `http://<pi>:8081` öffnen – laufen Live-Telegramme & Duty-Cycle ein?

Phase 4 - Namensauflösung (CCU/RaspberryMatic) · ~0,5 Tag

- In XS die Geräteliste-Quelle auf die CCU/RaspberryMatic (bzw. FHEM) konfigurieren.
- Prüfen, dass echte Gerätenamen statt Seriennummern erscheinen.
- Erste reale Analyse: Duty-Cycle-Top-Sender identifizieren, RSSI/Noise bewerten.

Phase 5 - Erweiterung: Metriken & Alarme · ~1-2 Tage

- Schwellwerte definieren (z. B. Duty-Cycle > X %, RSSI-Noise > Schwelle, „Dauersender“).
- Benachrichtigungen aufbauen (Telegram/E-Mail/Push über ioBroker oder Node-RED).
- Reboot-Loop-/Anomalie-Erkennung (auffällige Wiederholraten pro Gerät).

Phase 6 - Erweiterung: Langzeit-Logging & Dashboard · ~1-2 Tage

- InfluxDB + Grafana bereitstellen (nativ auf dem Pi oder via ioBroker-Adapter).
- Duty-Cycle/RSSI-Zeitreihen je Gerät schreiben.
- Grafana-Dashboard: Trend pro Gerät, Tages-/Wochen-Duty-Cycle, Störungs-Heatmap.

Phase 7 - Erweiterung: ioBroker-Integration · ~1-3 Tage

- Integrationsweg wählen (Node-RED / DB-Poll / eigener Adapter – Details in 9.3).
- Kennzahlen als ioBroker-States abbilden (Duty-Cycle, RSSI, letzte Aktivität je Gerät).
- Optional VIS-Kachel/Widget für den Funkstatus.

Phase 8 - Erweiterung: Hardware · optional, ~1-2 Tage

- Umstieg auf GPIO-Aufsteckplatine (Variante B) oder eigenes Gehäuse.
- Antennen-Optimierung (bessere 868-MHz-Antenne, Positionierung, ggf. Verlängerung).
- Optionales OLED am Sniffer für RSSI/Noise vor Ort.

- Perspektive: zweiter Sniffer an anderem Standort für Flächenabdeckung.

Phase 9 - Test, Abnahme & Doku · ~0,5-1 Tag

- Dauertest 1-2 Wochen; Vollständigkeit & Stabilität prüfen (Dienst-Neustartverhalten).
- Abnahmekriterien gegenprüfen (siehe 8.1).
- Kurz-Doku: Verkabelung, Ports, Config, Backup der SD-Karte/Container-Daten.

8.1 Abnahmekriterien (Definition of Done)

#	Kriterium
1	Sniffer empfängt stabil, XS zeigt Live-Telegramme.
2	Gerätenamen aus CCU/RaspberryMatic werden korrekt aufgelöst.
3	Duty-Cycle & RSSI je Gerät sind plausibel und werden angezeigt.
4	Dienst startet automatisch nach Reboot/Stromausfall.
5	Mindestens eine Erweiterung produktiv (Alarm ODER Grafana ODER ioBroker-States).
6	Konfiguration & Verkabelung dokumentiert, Datensicherung vorhanden.

9 Erweiterungskonzepte im Detail

9.1 Zusätzliche Metriken & Alarme

Ziel ist proaktives Erkennen von Funkproblemen, bevor Geräte ausfallen. Sinnvolle Regeln:

- **Duty-Cycle-Frühwarnung:** Alarm, wenn ein Gerät sich dem 1%-Limit nähert (z. B. > 0,7 % im gleitenden Stundenfenster).
- **Dauersender/Reboot-Loop:** Alarm bei ungewöhnlich hoher Telegrammrates oder periodischen Neustart-Mustern eines Geräts.
- **Funkqualität:** Alarm bei RSSI-Verschlechterung/Noise-Anstieg über Schwellwert.
- **Ausfall-/Stille-Erkennung:** Gerät sendet seit X Stunden nicht mehr (nur bei Geräten, die regelmäßig senden sinnvoll).

Umsetzung am einfachsten über die Integrationsebene (9.3): Kennzahlen in ioBroker/Node-RED auswerten und dort Telegram-/E-Mail-/Push-Benachrichtigungen auslösen.

9.2 Langzeit-Logging & Dashboard

AskSinAnalyzerXS besitzt eine eingebaute Langzeitaufzeichnung. Für flexible Auswertung empfiehlt sich zusätzlich eine Zeitreihen-DB:

- **InfluxDB** als Zeitreihen-Speicher (Duty-Cycle %, RSSI, Telegrammzähler je Gerät).
- **Grafana**-Dashboards: Duty-Cycle-Trend je Gerät, Tages-/Wochenvergleich, RSSI-Heatmap, Top-Sender-Ranking.
- Wenn die Werte ohnehin nach ioBroker fließen (9.3), ist das Logging via **ioBroker-InfluxDB-Adapter + Grafana-Adapter** mit wenig Aufwand erledigt.

Empfehlung

Datenweg *Sniffer* → *XS* → *ioBroker-States* → *InfluxDB* → *Grafana* wählen. So entsteht eine einzige, konsistente Datenquelle, und Alarme (9.1) und Dashboards (9.2) speisen sich aus denselben States.

9.3 ioBroker-Integration (Kernstück)

Da XS keine offiziell dokumentierte REST-/MQTT-Schnittstelle bereitstellt, gibt es drei realistische Integrationswege – nach Aufwand/Robustheit geordnet:

Weg	Idee	Aufwand / Eignung
1. Node-RED / RedMatic (empfohlen)	XS unterstützt RedMatic/Node-RED. Über den Live-Event-/WebSocket-Kanal die Telegramme in Node-RED (ioBroker <i>node-red</i> -Adapter) aufnehmen und in ioBroker-States schreiben.	Gering-mittel. Robust, wartbar, gut visualisierbar.
2. Datenbank-Poll	Die XS-Langzeitdaten periodisch auslesen und aggregiert nach ioBroker übertragen (z. B. per Script-Adapter).	Gering. Nur near-real-time, gut für Statistiken/Reports.
3. Eigener ioBroker-Adapter (Ausbaustufe)	Adapter, der den seriellen Sniffer-Strom selbst parst (paralleler zweiter Empfänger) oder XS-Daten konsumiert und native States/Objekte anlegt.	Höher. Sauberste Integration; passt zu deinem Adapter-Entwicklungs-Know-how.

Hinweis: Der genaue Live-Ausgabekanal (WebSocket/Event-Format) sollte zu Projektbeginn am laufenden System verifiziert werden, da er die Wahl zwischen Weg 1 und 3 bestimmt.

Beispiel-Zielstruktur der ioBroker-States

```
asksin.0.devices.<Geraetename>.dutyCycle // % im gleitenden Fenster
```

```
asksin.0.devices.<Geraetenname>.rssi           // dBm
asksin.0.devices.<Geraetenname>.lastSeen        // Zeitstempel
asksin.0.devices.<Geraetenname>.msgCount        // Telegramme/Stunde
asksin.0.info.noiseFloor                       // globales Grundrauschen
asksin.0.info.connection                       // Sniffer online?
```

9.4 Hardware-Erweiterung

- **Sauberer Dauereinbau:** Wechsel von Breadboard (A) auf GPIO-PCB (B) + Gehäuse.
- **Antenne:** hochwertige 868-MHz-Antenne, optimierte Position/Ausrichtung, Abstand zu Störquellen (Netzteile, USB3, Pi selbst).
- **Vor-Ort-Anzeige:** kleines OLED am Sniffer für RSSI/Noise ohne Weboberfläche.
- **Flächenabdeckung:** zweiter Sniffer an anderem Standort; Zusammenführung der Daten in ioBroker/Grafana zur Lokalisierung von Funklöchern.

10 Zeit- & Meilensteinplan

Phase	Meilenstein	Aufwand	Priorität
0	Entscheidung, Bestellung ausgelöst	~0,5 Tag	Muss
1	Sniffer empfängt (serielle Ausgabe ok)	~1 Tag	Muss
2	Pi-Basis & serielle Anbindung steht	~0,5 Tag	Muss
3	XS läuft, Live-Telegramme sichtbar	~0,5 Tag	Muss
4	Namensauflösung via CCU aktiv	~0,5 Tag	Muss
5	Erste Alarme produktiv	~1-2 Tage	Soll
6	Grafana-Dashboard steht	~1-2 Tage	Soll
7	ioBroker-States gefüllt	~1-3 Tage	Soll
8	Dauereinbau / Antennen-Optimierung	~1-2 Tage	Kann
9	Abnahme & Doku abgeschlossen	~0,5-1 Tag	Muss
Gesamt Grundaufbau (Phasen 0-4, 9)		ca. 3,5-4,5 Tage	
Gesamt inkl. aller Erweiterungen		ca. 8-14 Tage (verteilt)	

Aufwände sind Netto-Bastelzeit, realistisch verteilt auf mehrere Wochenenden. Kritischer Pfad = Phasen 1→2→3 (erst danach lässt sich alles Weitere testen).

11 Risiken & Gegenmaßnahmen

Risiko	Wahrsch.	Gegenmaßnahme
Netz ist überwiegend HmIP	abhängig	Vorab Anteil Klassik prüfen; Nutzen realistisch bewerten (Phase 0).
Schlechter Empfang / geringe Reichweite	mittel	Passende 868-MHz-Antenne, zentraler störungsarmer Standort, Abstand zum Pi.
Pi-UART-Konflikt (Konsole/Bluetooth)	mittel	Serielle Konsole deaktivieren, BT-UART umlegen (Phase 2); alternativ USB-UART (Variante A).
Falsche Baudrate / kein Datenstrom	mittel	Baudrate aus Sketch übernehmen, mit Terminal gegentesten.
3,3 V vs. 5 V Pegel am CC1101	gering	Konsequent 3,3-V-Board verwenden, keine 5-V-Signale.
XS-Live-Schnittstelle für ioBroker unklar	mittel	Am laufenden System verifizieren; Fallback: DB-Poll (Weg 2).
Bauteil-Verfügbarkeit	gering	Alternativen (Nano statt Pro Mini, andere CC1101-Boards) einplanen.

12 Quellen & Referenzen

- Haupt-Projekt (ESP32/Sniffer, Wiki): github.com/jp112sdl/AskSinAnalyzer
- Raspberry-Pi-taugliche Analyzer-Software: github.com/psi-4ward/AskSinAnalyzerXS
- npm-Paket: npmjs.com/package/asksin-analyzer-xs
- GPIO-Aufsteckplatine für den Pi: github.com/der-pw/AskSinAnalyzerXS-RPi
- Beispiel-PCB (Sniffer/Analyzer): github.com/stan23/myPCBs
- Wiki-Kernseiten: *Elektronik_Bauteile*, *Elektronik_Verdrahtung*, *Code*, *CCU_Unterstützung*, *WebUI*, *WLAN*, *Debugging*
- Hintergrund Duty-Cycle: HomeMatic-Forum & verdrahtet.info (Duty-Cycle-Probleme)